

BAZAR.COM

Lab 2 Report

REPLICATION	CACHING	CONSISTENCY
Catalog + Order replicas	Frontend in-memory cache	Invalidation on writes

Course	Distributed Operating Systems
Student Name	Ameer AboShams & Zeina Anabtawi
Student ID	12217464 & 12217834
Instructor	Dr. Samer Arandi
Date	24/5

01

Project Overview

Purpose, scope, and what changed from Lab 1.

This report presents the second part of the Bazar.com project. The system from Lab 1 was extended to support replication, load balancing, frontend caching, cache invalidation, and performance evaluation. The goal is to improve request latency while preserving consistent results after purchase and catalog update operations.

Lab 2 Goal

The main goal is to re-architect the original multi-tier online bookstore so it can handle higher workload using replication and caching while maintaining cache consistency.

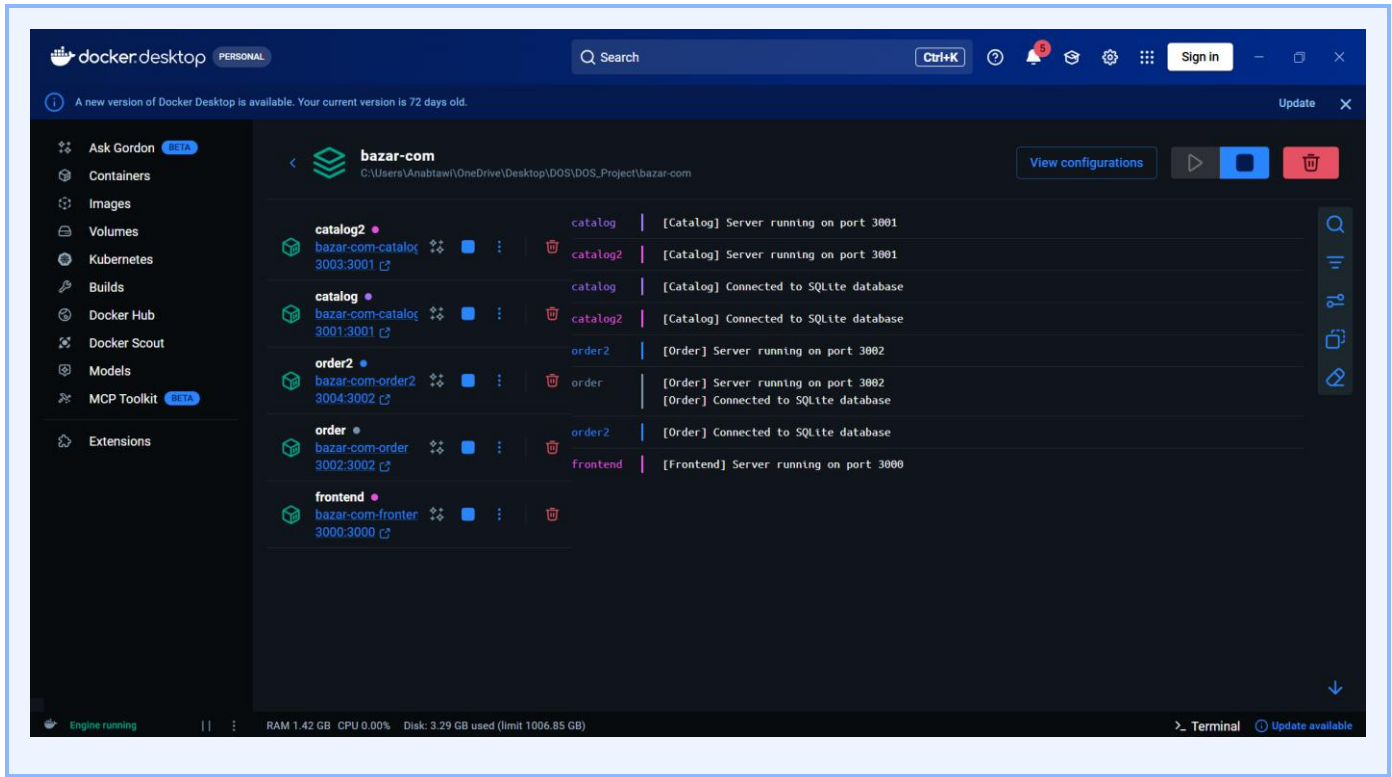
CATALOG New Catalog Items Three new books were added to the catalog, increasing the database from four items to seven items.	REPLICATION Replicated Services Two catalog replicas and two order replicas are deployed as separate containers.
CACHE Frontend Cache The frontend stores recently accessed book details in an in-memory cache.	CONSISTENCY Consistency Mechanism The cache is invalidated whenever a write operation changes stock or price data.
BALANCING Load balancing Round-robin selection inside frontend/server.js	PERFORMANCE Performance evaluation Postman response times with/without cache

02

System Architecture

Container layout and request flow.

The system keeps the frontend as the single entry point for all client requests. The frontend performs round-robin load balancing across replicated catalog and order services. The cache is integrated into the frontend process, which avoids an additional network hop for cached read requests.



Component	Container	Port	Responsibility
Frontend	bazar_frontend	3000	Client entry point, load balancing, caching, invalidation endpoint
Catalog Replica 1	bazar_catalog	3001	Query items and update stock/price
Catalog Replica 2	bazar_catalog2	3003 -> 3001	Second catalog replica for load balancing
Order Replica 1	bazar_order	3002	Processes purchase requests
Order Replica 2	bazar_order2	3004 -> 3002	Second order replica for load balancing
Database	bazar.db	SQLite file	Persistent storage for books and orders

Design simplification

In this implementation, the catalog and order replicas share the same SQLite database volume. This keeps data consistent and avoids divergence between replicas. A future version can use separate databases with an internal replication protocol.

docker-compose.yml excerpt - replicas and environment variables

```
services:
  catalog:
    build: ./catalog
    container_name: bazar_catalog
    ports:
      - "3001:3001"
    environment:
      - DB_PATH=/app/data/bazar.db
      - FRONTEND_URL=http://frontend:3000
    volumes:
      - ./data:/app/data

  catalog2:
    build: ./catalog
    container_name: bazar_catalog2
    ports:
      - "3003:3001"
    environment:
      - DB_PATH=/app/data/bazar.db
      - FRONTEND_URL=http://frontend:3000
    volumes:
      - ./data:/app/data

  order:
    build: ./order
    container_name: bazar_order
    ports:
      - "3002:3002"
    environment:
      - DB_PATH=/app/data/bazar.db
      - CATALOG_URL=http://catalog:3001
      - FRONTEND_URL=http://frontend:3000

  order2:
    build: ./order
    container_name: bazar_order2
    ports:
      - "3004:3002"
    environment:
      - DB_PATH=/app/data/bazar.db
      - CATALOG_URL=http://catalog2:3001
      - FRONTEND_URL=http://frontend:3000
```

03

Implementation Details

Major changes added in Lab 2.

Feature	Implementation	How it is verified
Replication	docker-compose defines catalog2 and order2 in addition to the original catalog and order services.	Docker Desktop / docker compose logs show five containers running.
Load Balancing	The frontend uses round-robin indexes to alternate catalog and order requests between replicas.	Responses contain servedBy: catalog or servedBy: catalog2.
Caching	The frontend stores /info/:id results inside an in-memory Map.	First request is MISS; second request is HIT.
Invalidation	Order and catalog services call frontend /invalidate/:id before write operations.	Cache is cleared after purchase, stock update, or price update.
Persistence	SQLite stores books and order records on disk.	Database viewer shows books and orders tables.

04

Load Balancing

Round-robin behavior

The frontend stores a list of catalog replicas and order replicas. For each new request, it picks the next replica in the list. This distributes requests between the two replicas in a simple and predictable way.

frontend/server.js excerpt - replica lists and round-robin functions

```
const CATALOG_REPLICAS = [
  process.env.CATALOG_URL || 'http://localhost:3001',
  process.env.CATALOG_URL_2 || 'http://localhost:3003'
];

const ORDER_REPLICAS = [
  process.env.ORDER_URL || 'http://localhost:3002',
  process.env.ORDER_URL_2 || 'http://localhost:3004'
];

let catalogIndex = 0;
let orderIndex = 0;
```

```

function getNextCatalogReplica() {
  const replica = CATALOG_REPLICAS[catalogIndex];
  catalogIndex = (catalogIndex + 1) % CATALOG_REPLICAS.length;
  console.log(`[Frontend] Forwarding catalog request to: ${replica}`);
  return replica;
}

function getNextOrderReplica() {
  const replica = ORDER_REPLICAS[orderIndex];
  orderIndex = (orderIndex + 1) % ORDER_REPLICAS.length;
  console.log(`[Frontend] Forwarding order request to: ${replica}`);
  return replica;
}

```

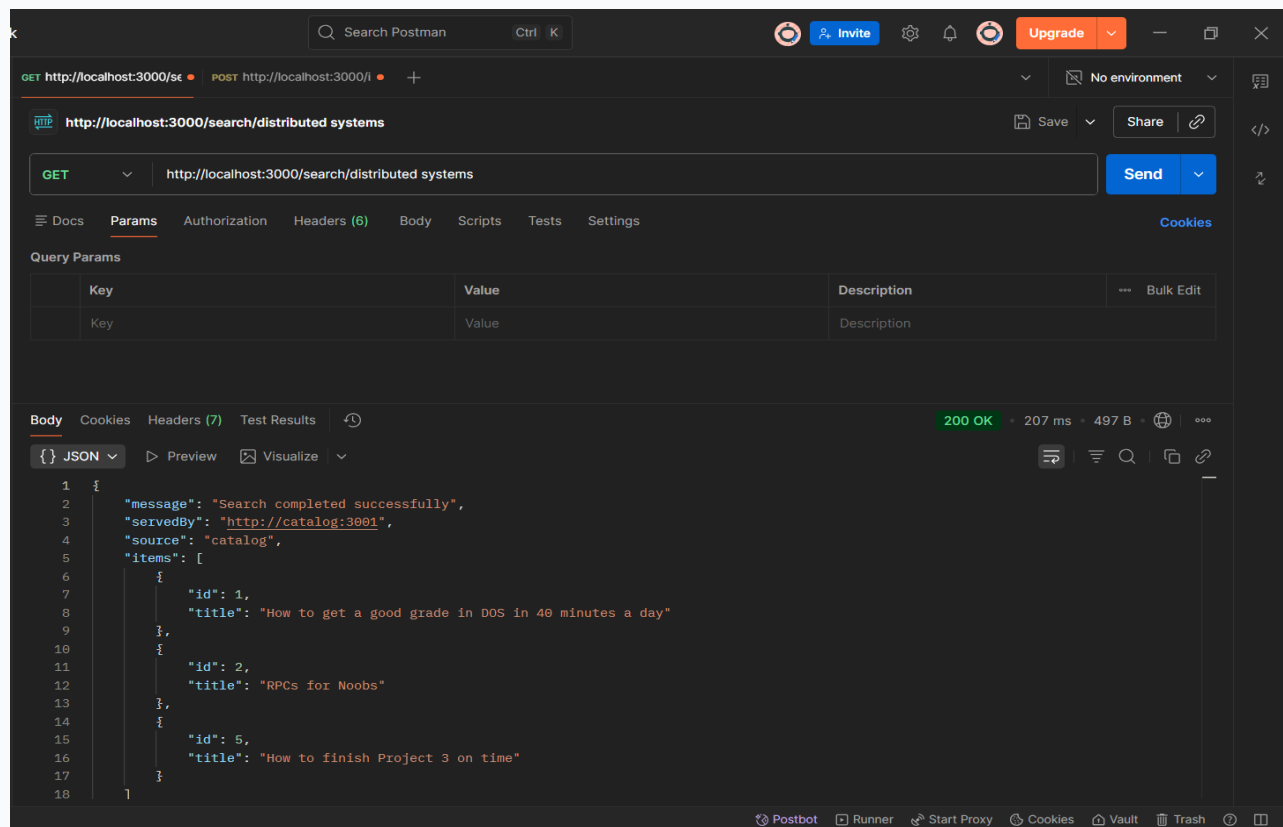


Figure 4. Load balancing output: request served by catalog replica 1.

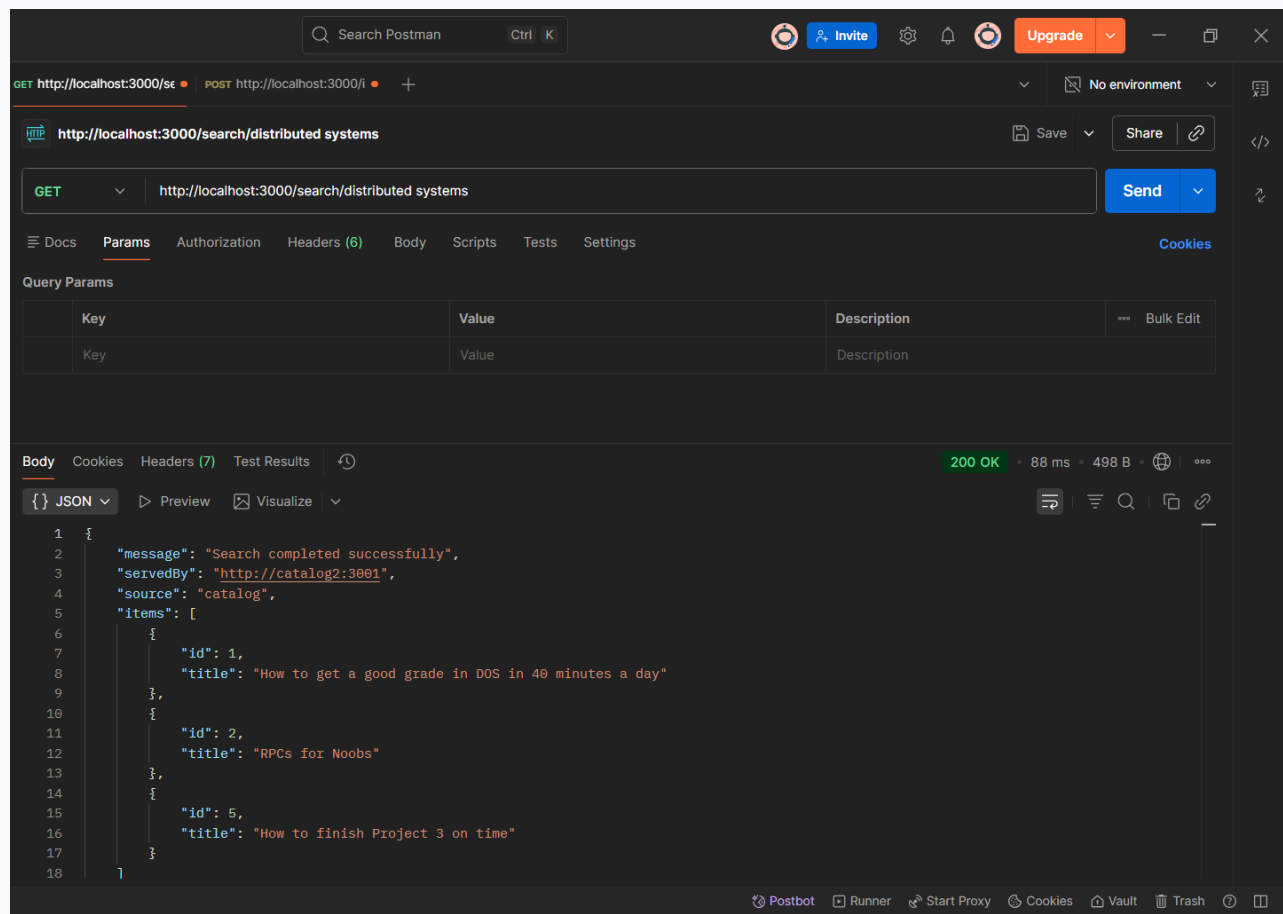


Figure 5. Load balancing output: request served by catalog replica 2.

05 Frontend Cache

Caching idea

The cache is implemented as an in-memory Map inside the frontend service. It stores results of `/info/:id` requests. If the requested item is already stored, the frontend returns it directly from memory. Otherwise, it forwards the request to a catalog replica and stores the result.

frontend/server.js excerpt - cache map and cache endpoint

```
const itemCache = new Map();

app.get('/cache', (req, res) => {
  const cacheContent = Object.fromEntries(itemCache);

  res.json({
    message: 'Current frontend cache content',
    cacheSize: itemCache.size,
    cache: cacheContent
  });
});
```

frontend/server.js excerpt - cache HIT and MISS logic

```
app.get('/info/:id', async (req, res) => {
  const id = req.params.id;

  if (itemCache.has(id)) {
    console.log(`[Frontend] CACHE HIT for item: ${id}`);
    return res.json({
      message: 'Item details retrieved successfully from cache',
      source: 'cache',
      cacheStatus: 'HIT',
      item: itemCache.get(id)
    });
  }

  console.log(`[Frontend] CACHE MISS for item: ${id}`);
  const catalogUrl = getNextCatalogReplica();
  const response = await axios.get(`${catalogUrl}/query/item/${id}`);
  const item = response.data.item;
  itemCache.set(id, item);

  res.json({
    message: 'Item details retrieved successfully from catalog',
    servedBy: catalogUrl,
    source: 'catalog',
    cacheStatus: 'MISS',
    item
  });
});
```

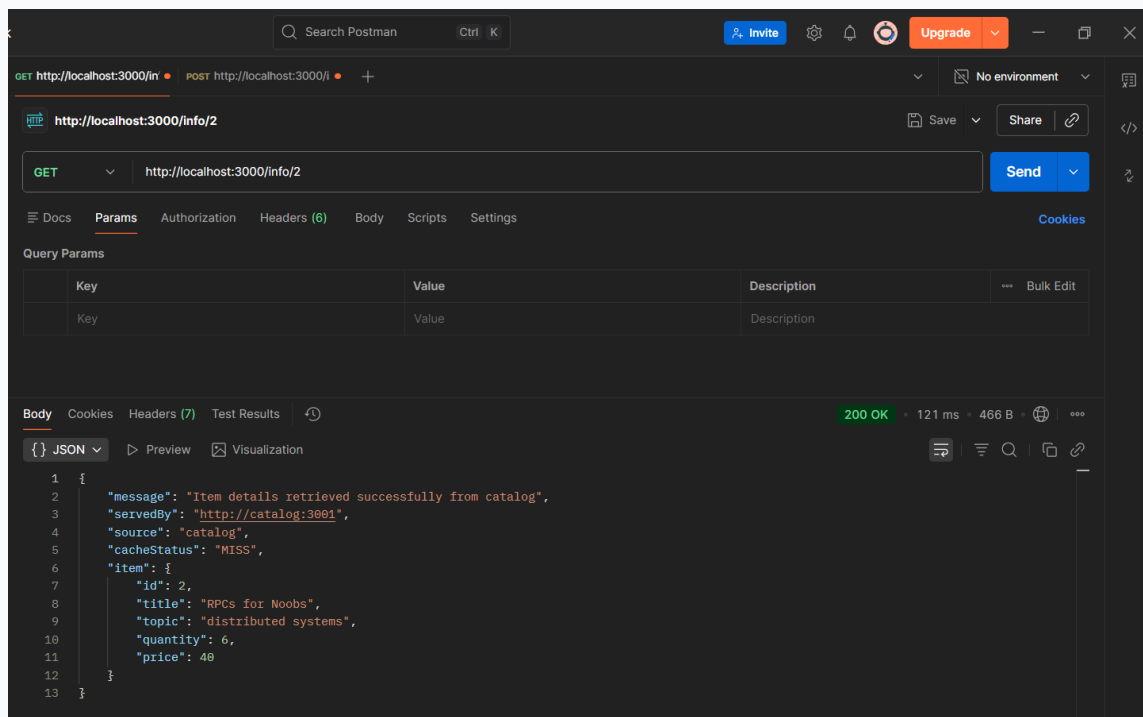



Figure 6. First GET /info/2 request showing CACHE MISS.

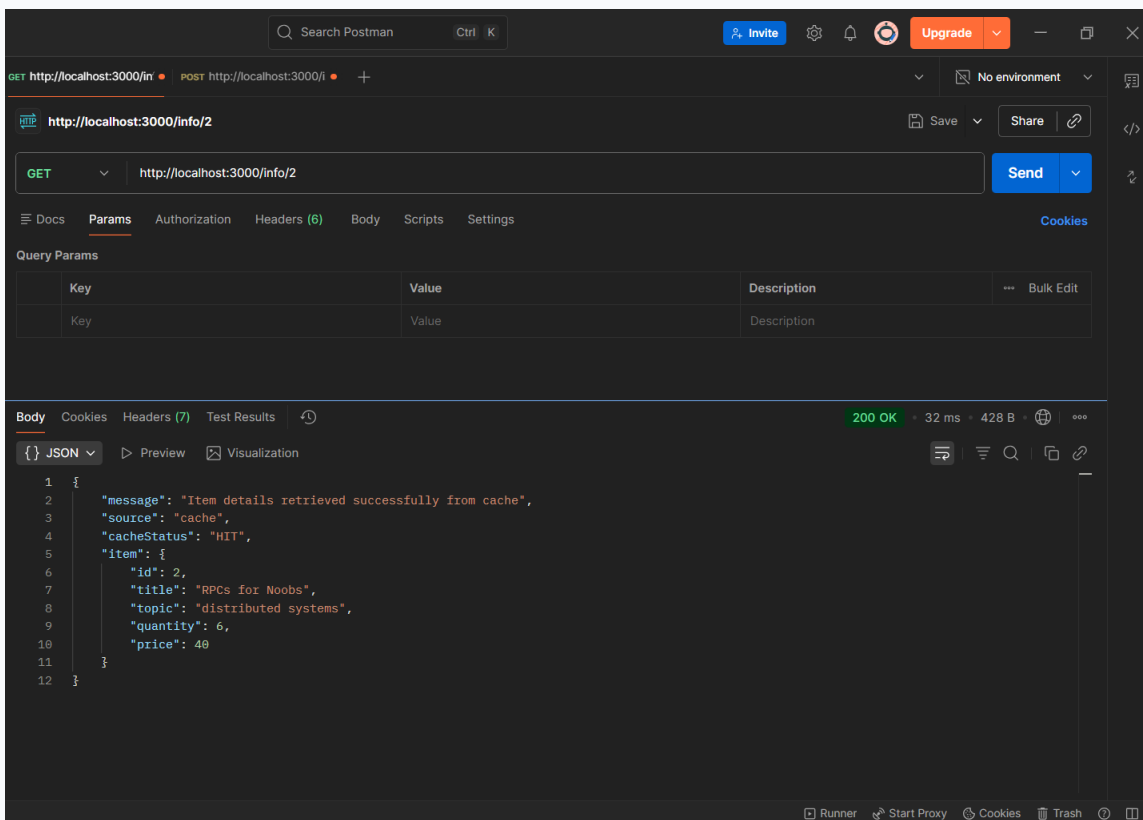


Figure 7. Second GET /info/2 request showing CACHE HIT.

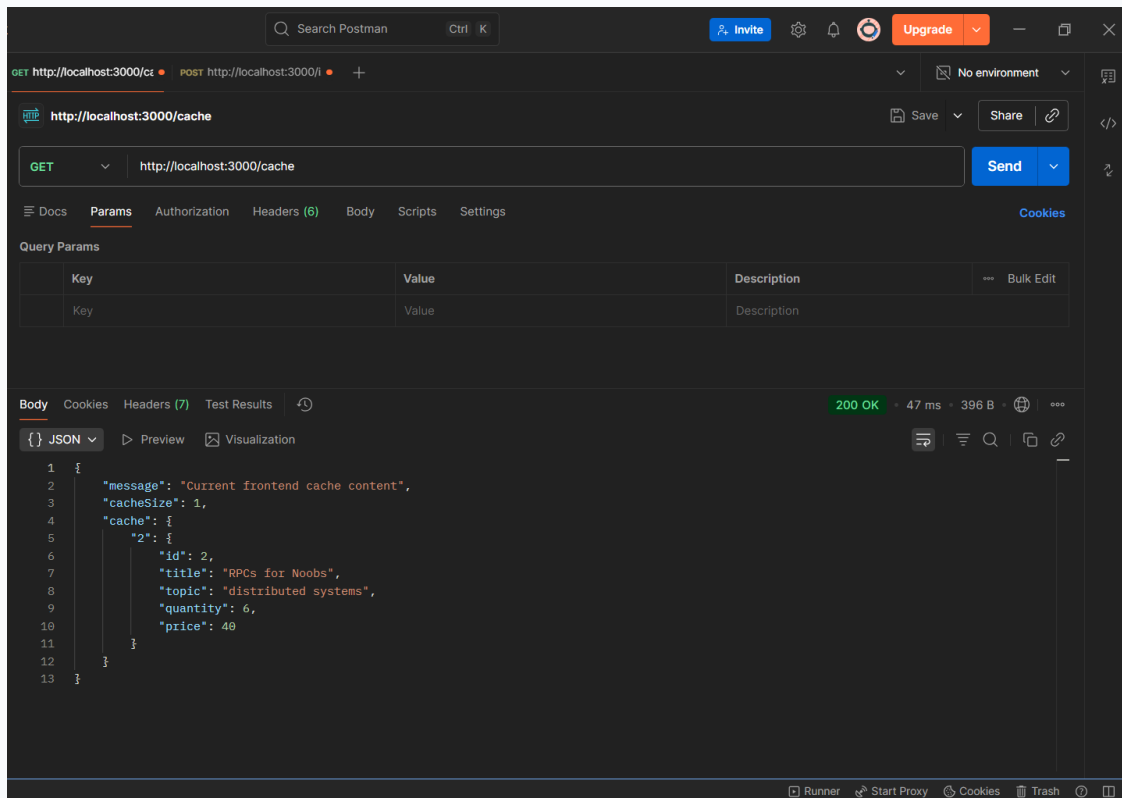


Figure 8. GET /cache showing item 2 stored in the frontend cache.

06 Cache Invalidation and Consistency

Why invalidation is needed

Read results can become stale after a purchase, stock update, or price update. To keep the cache consistent, the backend sends an invalidate request to the frontend before writing to the database. The frontend then removes the cached item.

frontend/server.js excerpt - invalidate one item

```

app.post('/invalidate/:id', (req, res) => {
  const id = req.params.id;

  if (itemCache.has(id)) {
    itemCache.delete(id);
    console.log(`[Frontend] Cache invalidated for item: ${id}`);
  }
});

```

```

    return res.json({
      message: `Cache invalidated for item ${id}`,
      itemId: id
    });
  }

  res.json({
    message: `No cache entry found for item ${id}`,
    itemId: id
  });
});

```

order/server.js excerpt - invalidate cache before purchase write

```

const FRONTEND_URL = process.env.FRONTEND_URL || 'http://frontend:3000';

async function invalidateFrontendCache(itemId) {
  try {
    await axios.post(`${FRONTEND_URL}/invalidate/${itemId}`);
    console.log(`[Order] Cache invalidation request sent for item ${itemId}`);
  } catch (error) {
    console.warn(`[Order] Cache invalidation failed for item ${itemId}: ${error.message}`);
  }
}

// inside purchase endpoint, before stock update:
await invalidateFrontendCache(id);

const updateResponse = await axios.post(`${CATALOG_URL}/update/stock/${id}`, {
  change: -quantity
});

```

catalog/server.js excerpt - invalidate cache before catalog update

```

const FRONTEND_URL = process.env.FRONTEND_URL || 'http://frontend:3000';

async function invalidateFrontendCache(itemId) {
  try {
    await axios.post(`${FRONTEND_URL}/invalidate/${itemId}`);
    console.log(`[Catalog] Cache invalidation request sent for item ${itemId}`);
  } catch (error) {
    console.warn(`[Catalog] Cache invalidation failed for item ${itemId}: ${error.message}`);
  }
}

// before UPDATE books SET quantity = ? WHERE id = ?
await invalidateFrontendCache(id);

```

Testing flow

The following screenshots should be added after running each request in Postman. Each screenshot should clearly show the URL, method, status code, and important fields such as `servedBy`, `cacheStatus`, `cacheInvalidated`, or `cacheSize`.

Step	Request	Expected Result
1	GET /info/2	First request returns <code>cacheStatus = MISS</code>
2	GET /info/2 again	Second request returns <code>cacheStatus = HIT</code>
3	GET /cache	Item 2 appears in frontend cache
4	POST /purchase/2	<code>cacheInvalidated = true</code> and stock decreases
5	GET /cache	Item 2 removed from cache
6	GET /info/2	<code>cacheStatus = MISS</code> and fresh data is returned
7	POST /update/stock/2	<code>cacheInvalidated = true</code> after catalog update

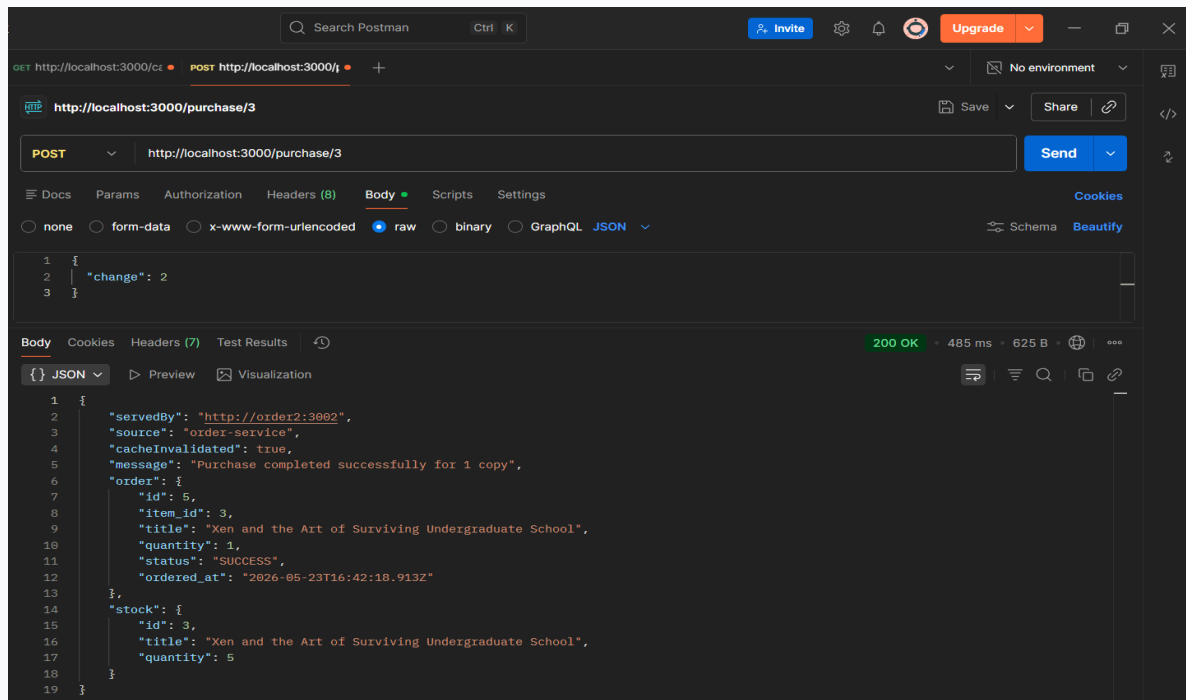


Figure 9. Purchase request showing `cacheInvalidated = true`.

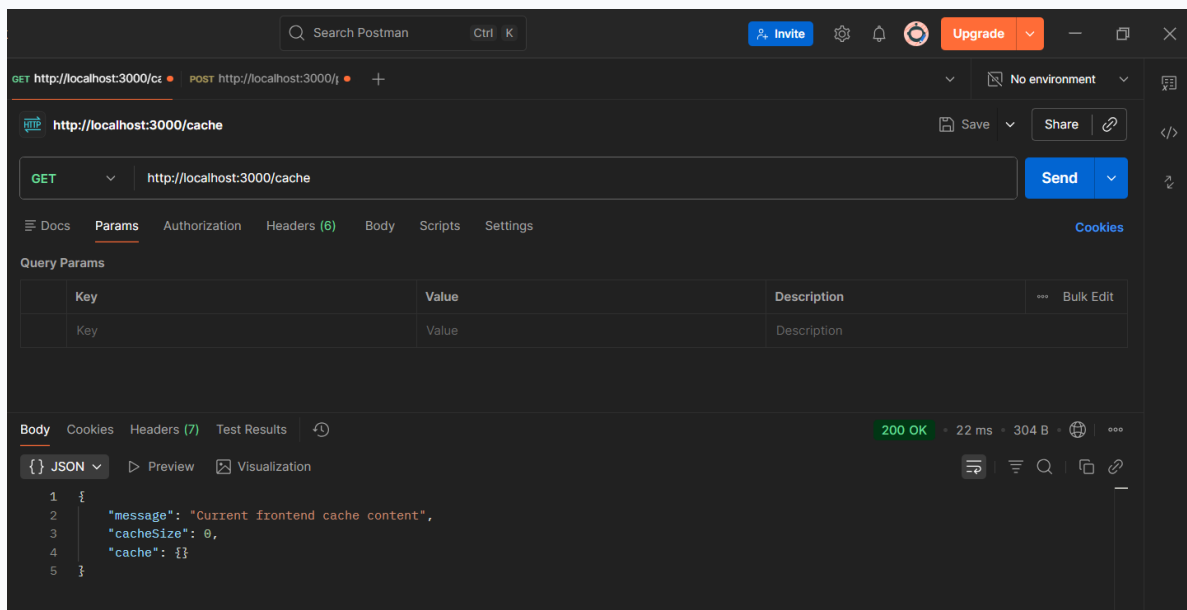


Figure 10. Cache after purchase showing the item removed.

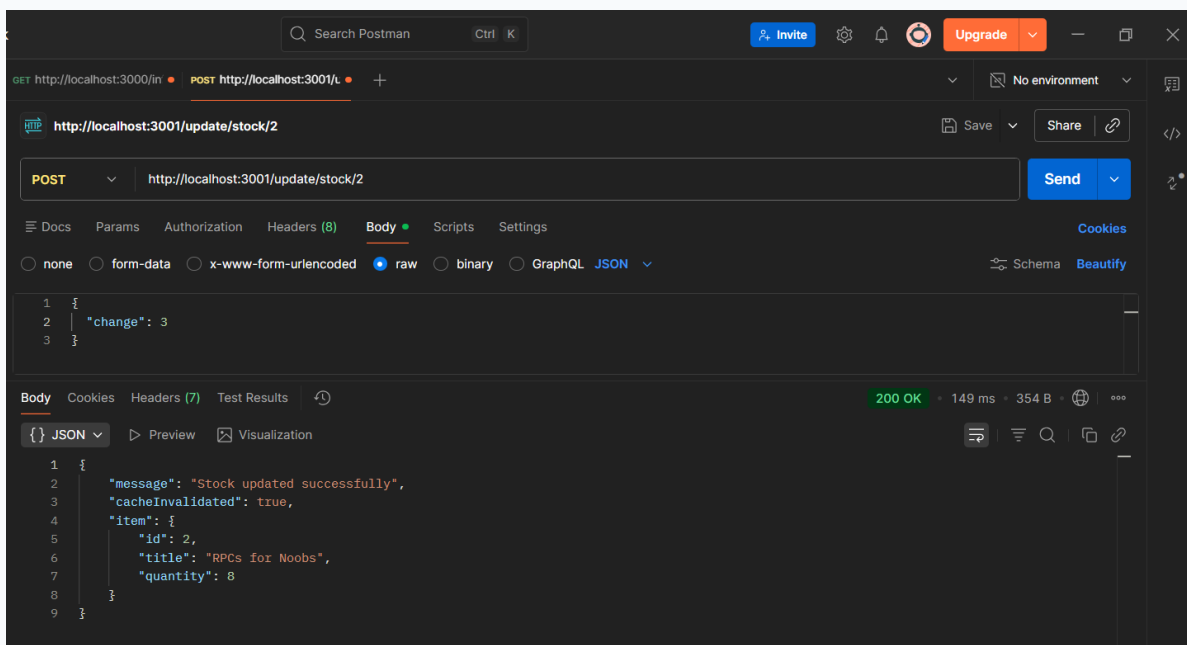


Figure 11. Catalog stock update showing `cacheInvalidated = true`.

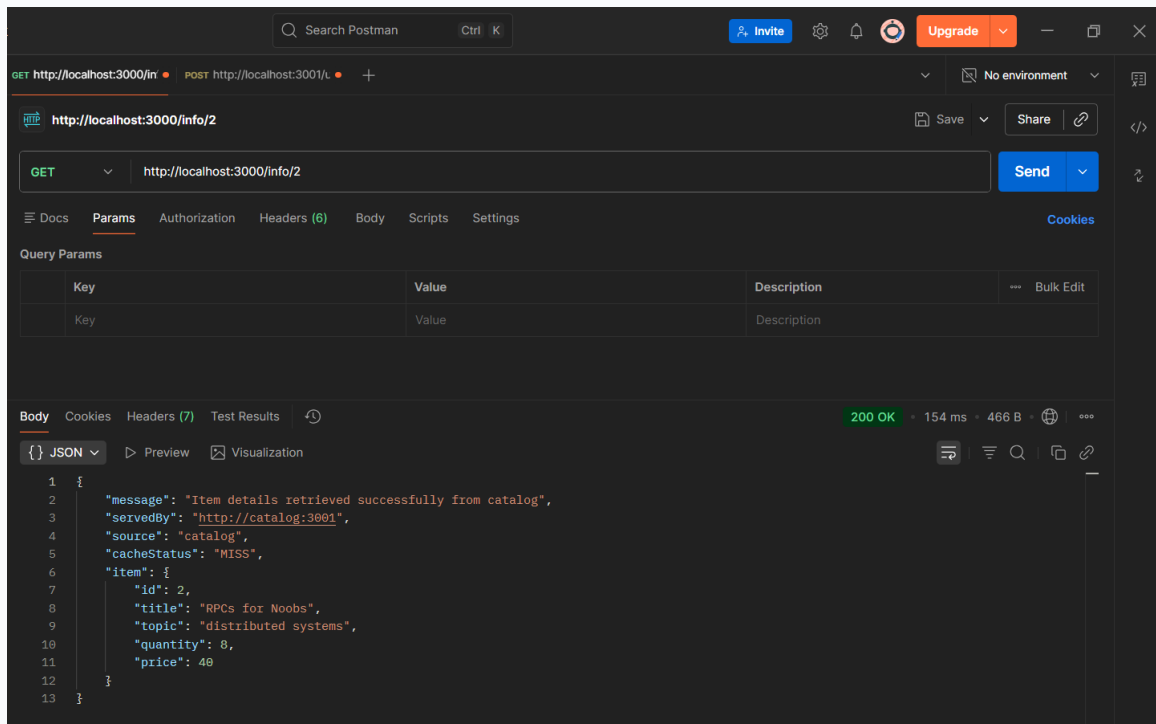


Figure 12. Request after invalidation showing CACHE MISS and fresh data.

08 Performance Evaluation

Response-time measurements and analysis.

The following table should be filled with measured response times from Postman. Each request can be executed three times and averaged. The expected result is that cache HIT requests should be faster than cache MISS requests because they are served locally by the frontend without contacting the catalog replicas.

Test Case	Run 1	Run 2	Run 3	Average	Notes
GET /info/2 - Cache MISS	49ms	17ms	12ms	26ms	Catalog request
GET /info/2 - Cache HIT	12ms	8ms	10ms	10ms	Served from cache
POST /purchase/2	212ms	167 ms	75ms	151ms	Write request
GET /info/2 after purchase invalidation	30ms	17ms	16ms	21ms	MISS after invalidation
POST /update/stock/2	33ms	39 ms	33ms	35ms	Catalog write
GET /info/2 after catalog update	17ms	19ms	15ms	17ms	Fresh data

The second part of Bazar.com successfully extends the Lab 1 system with replication, load balancing, caching, and consistency handling. The frontend now balances read and purchase requests across replicas and stores recently accessed item details in memory. Cache invalidation is triggered before write operations, which prevents stale item data after purchases and catalog updates. Performance measurements can be used to show that cached reads reduce response time compared with cache misses.